

Scalable Multi-Robot Cooperation for Multi-Goal Tasks Using Reinforcement Learning

Tianxu An , Joonho Lee , Marko Bjelonic , Flavio De Vincenti , and Marco Hutter , *Member, IEEE*

Abstract—Coordinated navigation of an arbitrary number of robots to an arbitrary number of goals is a big challenge in robotics, often hindered by scalability limitations of existing strategies. This letter introduces a decentralized multi-agent control system using neural network policies trained in simulation. By leveraging permutation invariant neural network architectures and model-free reinforcement learning, our policy enables robots to prioritize varying numbers of collaborating robots and goals in a zero-shot manner without being biased by ordering or limited by a fixed capacity. We validate the task performance and scalability of our policies through experiments in both simulation and real-world settings. Our approach achieves a 10.3% higher success rate in collaborative navigation tasks compared to a policy without a permutation invariant encoder. Additionally, it finds near-optimal solutions for multi-robot navigation problems while being two orders of magnitude faster than an optimization-based centralized controller. We deploy our multi-goal navigation policies on two wheeled-legged quadrupedal robots, which successfully complete a series of multi-goal navigation missions.

Index Terms—Legged locomotion, multi-robot systems, reinforcement learning.

I. INTRODUCTION

HUMANS routinely deal with multiple collaborators and goals simultaneously. For example, a group of friends splits to find various products in a grocery store. In this scenario, individuals naturally decide with whom to collaborate, and which product to find first. Can robots have similar capabilities? Despite recent advances that have made robots proficient in tasks like locomotion [1], [2], [3] and navigation [4], [5], these approaches cannot be directly applied to cooperative multi-robot multi-goal (MRMG) tasks with arbitrary numbers of robots and goals.

Received 7 August 2024; accepted 3 December 2024. Date of publication 20 December 2024; date of current version 7 January 2025. This article was recommended for publication by Associate Editor H. Chen and Editor A. Kheddar upon evaluation of the reviewers' comments. This work was supported in part by Swiss National Science Foundation through the National Centre of Competence in Digital Fabrication (NCCR dfab) and the European Research Council under Grant 852044, Grant 101070596, and Grant 101121321. (Tianxu An and Joonho Lee contributed equally to this work.) (Corresponding author: Joonho Lee.)

Tianxu An and Marco Hutter are with Robotic Systems Lab, 8092 Zürich, Switzerland.

Joonho Lee is with Neuromeka Company, Ltd., Seoul 04782, South Korea (e-mail: joonho.lee@neuromeka.com).

Marko Bjelonic is with Swiss-Mile Robotics AG, 8092 Zürich, Switzerland.

Flavio De Vincenti is with Computational Robotics Lab, ETH Zürich, 8092 Zurich, Switzerland.

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2024.3521183>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3521183

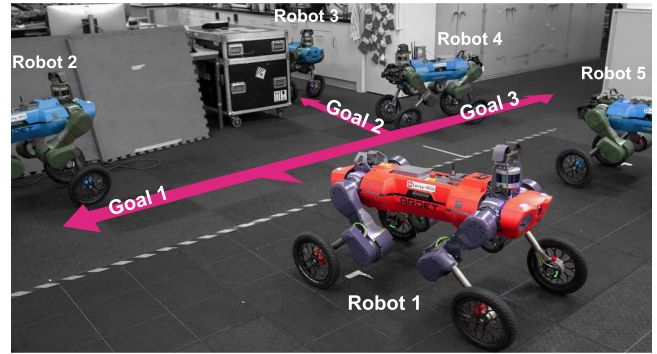


Fig. 1. Multi-robot multi-goal (MRMG) navigation problem. Multiple wheeled-legged robots are given multiple goal locations to visit. Robots must decide on goal assignments among themselves and conduct cooperative path planning to complete the task in the shortest time.

A gap remains in handling flexible numbers of robots and goals in high-dimensional real-world settings. Several prior works demonstrated decentralized agents that process state information from a flexible number of collaborators [6], [7], [8], [9], [10]. However, most existing works focus on theoretical analysis for simple 2D problems in simulation and restrict the interactions between robots and goals through predefined orderings.

We focus on developing a scalable decentralized control policy for high-dimensional, partially observable MRMG problems depicted in Fig. 1. Each robot relies only on the local perception and neighbor information, and each task episode may include a different number of robots and goals. To adapt the system to a dynamic real-world environment, each agent must dynamically prioritize goals and neighbors to interact with: each robot can decide which goals to complete and dynamically attend to specific neighbors to avoid collision or facilitate collaborative interactions.

Many existing multi-agent control approaches are limited in addressing this problem. Firstly, model-based optimization methods struggle to scale to higher numbers of robots [11], [12], [13] because the number of decision variables can grow arbitrarily with the number of robots and goals, leading to solver failure or high computational cost. Recent works predominantly focus on decentralized reinforcement learning (RL) policies to overcome scalability issues. However, directing policies' prioritization on neighbors or goals is still not straightforward. Manually commanding each robot to focus on fixed collaborators or goals can lead to sub-optimality [14], [15]. An implicit prioritization

can be learned by concatenating all states of collaborators and goals into a single observation [16], [17], but this approach does not scale to higher numbers. To extend such implicit prioritization to an arbitrary number of collaborators or goals, [6], [18] employ LSTM networks to receive state information from all robots and goals while maintaining the dimension of the output features regardless of the number. Nevertheless, using LSTM as the aggregator necessitates manual ordering of neighbors and goals based on hand-engineered heuristics. Earlier states of neighbors or goals could be forgotten, resulting in performance loss when many robots and goals exist.

We adopt permutation invariant neural networks to construct RL policies for the MRMG problems. Existing works have applied permutation invariant network structures to encode the states of neighboring agents in 2D multi-agent problems [8], [9], [10]. Employing an architecture similar to [10] that encodes robot states with PointNet [19] structures, we utilize permutation invariant neural networks to encode the states of arbitrary numbers of collaborating robots and goals.

We validate our approach on the MRMG navigation experiment in Fig. 1 and evaluate each component in simulation. Our results demonstrate that the permutation invariant policy can effectively scale to arbitrary numbers of robots and goals and beyond the numbers that are initially trained with.

Our main contribution lies in developing a complete MRMG navigation system for quadrupedal robots as depicted in Fig. 1. We have integrated the permutation invariant policies with robot-to-robot communication based on the gRPC framework [20]. Our key contributions are as follows:

- 1) We introduce an RL-based approach to MRMG problems for robots, specifically for navigation problems for multiple quadrupedal robots. Our system leverages permutation invariant neural networks, which allow decentralized agents to learn a dynamic prioritization of flexible numbers of collaborators and goals.
- 2) We verify the effectiveness of our system through real-world MRMG navigation experiments with two wheeled-legged quadrupedal robots [21]. We provide important technical details regarding training and hardware implementation for inter-robot communication.

II. RELATED WORK

A. Permutation Invariant Neural Networks

Permutation invariant neural networks ensure output invariance to permutations of the input data. This is useful for RL agents to process flexible numbers of neighbors and goals observed in arbitrary orders. Previous works have applied mean-field approximation [7], [22], self-attention encoders [23], [24], pooling operations [9], [10], [25], and explicit constraints [26] to ensure the permutation invariant properties of their networks. Our network architecture is inspired by [10], which uses a max-pooling operation similar to PointNet [19]. This architecture is simple, fast, and memory-efficient compared to other methods. More importantly, the PointNet [19] architecture helps decentralized agents learn a prioritization strategy while effectively ensuring permutation invariance of neighbors and goals in the ego robots' observation space.

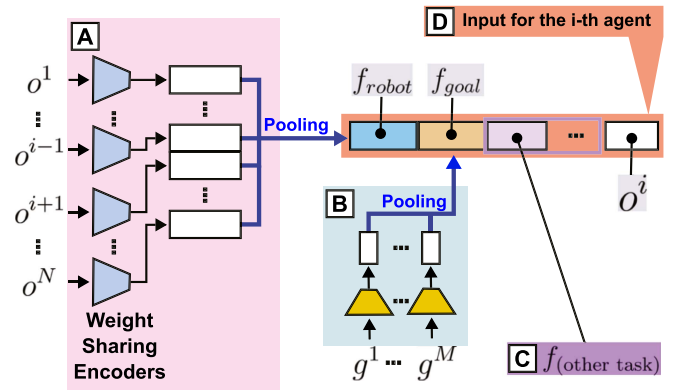


Fig. 2. Encoding of high-level policy input for the i -th robot. (A) all neighboring robot states are passed to a shared encoder and pooled to generate fixed size feature vector. (B) The same operation is done for the goal states, using different weights. (C) The feature space can be extended with new kinds of inputs (e.g., obstacles or opponents) depending on the task. (D) The concatenated vector of encoded features from (A, B, C) and the local observation O^i is the input to the high-level policy for the i -th robot.

B. Multi-Goal Navigation

Multi-goal navigation tasks require robots to reason about inter-goal relationships to plan a navigation sequence. For single-robot multi-goal navigation, [27] greedily plans the next navigation goal, while [5] computes an offline path before execution. Both methods may get stuck if a goal point is unreachable in execution time, as common in real-world scenarios, due to improper goal setup or unexpected map changes. Recently, it has become more popular to control multiple robots to increase efficiency in multi-goal navigation. For MRMG navigation tasks, agents need to conduct cooperative path planning besides goal assignment. Although the two problems can be handled separately [14], [28], more recent works apply multi-agent reinforcement learning (MARL) algorithms to couple them into a single problem [6], [29]. Another line of MRMG navigation tasks aims to achieve object manipulation through robot navigation [29], [30], where manipulating objects is essentially seen as robots navigating to implicit goals based on the locations of objects and packing sites. Similar to these MARL approaches, our policies prioritize and distribute goals while synchronously planning robot motion to avoid collisions with neighbors. We also apply our approach to a multi-object manipulation task where robots are asked to push a flexible number of objects into a packing site.

III. METHOD

A. Overview

Our MARL pipeline adopts a hierarchical architecture, similar to [30], [31], where high-level policies produce position or velocity commands to the low-level policy that is pre-trained and fixed. All MRMG problems in this letter are solved with task-specific high-level policies but the same low-level policy, and we refer readers to [31] for its technical details.

Our high-level policies receive input from the encoding pipeline shown in Fig. 2 with its details described in Section III-C. The high-level policies output either target body

xy positions in the robot frame, or target body velocities consisting of linear xy velocity and yaw rates in the robot frame, depending on the task. The encoders and the high-level policy are trained end-to-end using a PPO algorithm [32]. To ensure policy outputs adhere to the constraints of our robot's physical capabilities, we adopt a bounded action space based on the Beta distribution [31], [33].

It is worth noting that our methods of training high-level policies can be expanded to other robot morphologies as the high-level policy structure is independent of morphologies.

For real-world experiments, we establish a robot communication pipeline using the gRPC framework [20], as described in Section III-E.

B. Problem Formulation

We model the MRMG problems as a decentralized partially observable Markov decision process (Dec-POMDP). A Dec-POMDP is defined by a tuple: $\langle \mathcal{A}, \mathcal{S}, \mathcal{U}, \Omega, \mathcal{O}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{A}$ denotes the set of N agents (robots) $\mathcal{A} = \{a^i\}_{i=1}^N$, and $s \in \mathcal{S}$ is the global state of the environment. Unlike previous works [17], [30], where the number of robots N is fixed for all environments, we allow N to be a random system state. Since we use homogeneous robots, they all share the same action space $\mathcal{U}$ and observation space Ω . We use $\mathbf{o} \in \Omega^N$ and $\mathbf{u} \in \mathcal{U}^N$ to represent the joint observations and joint actions of all robots, respectively: $\mathbf{o} = (o^1, \dots, o^N)$ and $\mathbf{u} = (u^1, \dots, u^N)$. Due to partial observability, each agent a^i cannot observe the full global state s_t at time step t , but a partial observation state $o_t^i \in \Omega$ from the observation function $O(s_t, u^i)$. $\mathcal{P}$ denotes the transition dynamics, with $\mathcal{P}(s_{t+1}|s_t, \mathbf{u}_t)$ being the probability of transitioning to the next global state s_{t+1} by taking the joint action $\mathbf{u}$ at global state s_t . $\mathcal{R}: \mathcal{S} \times \mathcal{U} \rightarrow \mathbb{R}$ is the common reward function shared by all robots, and $\gamma \in [0, 1]$ is the discount factor. The robots collectively roll out a trajectory $\tau = (s_0, \mathbf{u}_0, s_1, \dots)$ based on the shared decentralized policy $\pi(u^i|o^i): \Omega \times \mathcal{U} \rightarrow [0, 1]$. The goal of all robots is to find the best policy to maximize the cumulative discounted reward $\mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, \mathbf{u}_t)]$.

C. Permutation Invariant Encoder

We let each robot a^i observe all collaborators and goals and enable them to learn to prioritize. We cannot simply concatenate states all collaborators and all goals as part of the ego robot's observation because N and M are not fixed and the orders should not affect the policy output.

Using permutation invariant encoders, we can obtain a fixed-size feature vector f_{robot} for all collaborators and f_{goal} for all goals, as shown in Fig. 2. The encoder leverages the pooling operation similarly to PointNet [19]. For each robot a^i , the observations of all collaborators are passed to a weight-sharing MLP to get corresponding local robot features; then, all per-robot features are max-pooled to obtain the fixed-length feature f_{robot} (Fig. 2(A)). The same operation is performed for the M goals with separate encoders and pooling operation (Fig. 2(B)). Both f_{robot} and f_{goal} are concatenated with ego robot observation o^i (Fig. 2(D)), including robot-centered proprioceptive and exteroceptive measurements. The concatenated vector is provided to

TABLE I
REWARDS

MRMG Navigation		
Reward	Description	Scale
Termination	1.0 if game success	10.0
Distance to Goal	$\exp(-(\text{distance to unreached goals})^2)$	5.0
Motion bonus	$\text{clip}(\text{moving speed}, 0.0, 1.0)$	1.0
Neighbor distance	-1.0 if too close (< 1 m)	1.0
Collision	-1.0 if robot collides	2.0

Box Packing		
Reward	Description	Scale
Termination	1.0 if game success	5.0
Progress	$N_{\text{completed boxes}}/N_{\text{total box count}}$	0.25
Box velocity	$\sum_{\text{boxes}} (v_{\text{box}} \cdot \text{direction to goal})$	0.1
Box position	$\sum_{\text{boxes}} \exp(-(p_{\text{box}} - p_{\text{goal}})^2)$	0.5
Neighbor distance	-1.0 if too close (< 1 m)	1.0

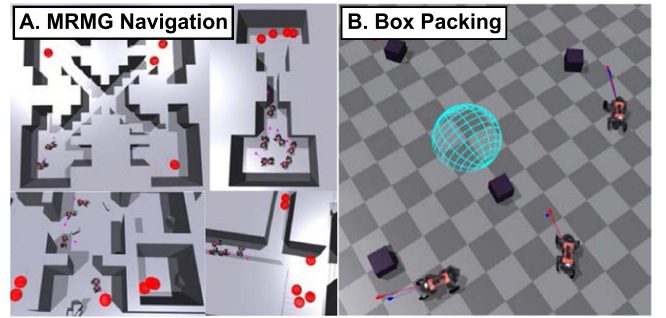


Fig. 3. Training environments (A) MRMG Navigation tasks in different training environments. Goals are displayed as red spheres. (B) In Box Packing environments, robots push boxes to the blue packing site.

the downstream high-level policy network as input. For problems involving additional task-related states (e.g., manipulable objects) of variable numbers, we can introduce additional global features $f_{(\text{other task})}$ (Fig. 2(C)).

D. Training Environments

Here we describe the training environments for the two MRMG problems in this letter. Both problems have a time limit of 45s and the numbers of robots, goals, and objects are randomized per episode. The rewards for training the two policies are displayed in Table I.

1) *MRMG Navigation*: A group of robots is asked to visit multiple goal locations. Each goal has to be visited at least once by any robots. We represent each goal as a 0.5 m radius sphere and initialize it 1530 m away from the centroid of the robot group. An episode ends successfully when all goals are reached, or fails if the time limit is exceeded. We design different terrains as shown in Fig. 3(A). During training, each episode involves up to four robots and four goals, with the number of robots and goals varying across episodes. Note that in policy deployment, the numbers can be higher than the maximum values in the training stage.

A terrain curriculum similar to [2] is used using procedurally generated terrains by [31]. The terrain becomes more difficult if a task is completed successfully, or becomes easier if no goal is reached. Terrain difficulty is associated with handcrafted

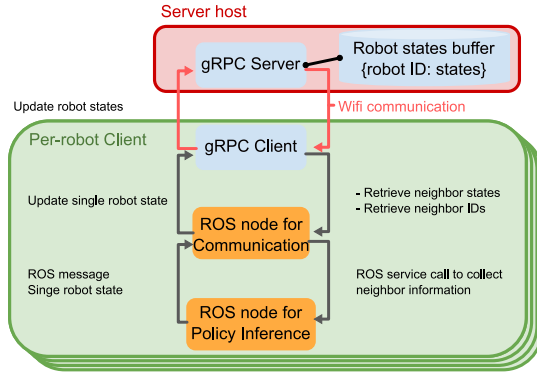


Fig. 4. gRPC communication framework. A gRPC server contains a message buffer with ID-state pairs. Each robot has a unique ID used for robot identification when communicating with clients.

attributes, such as the narrowness of corridors, ground flatness, height of stairs, etc.

2) *Box Packing*: A group of robots is asked to push multiple boxes to a packing site, as shown in Fig. 3(B). The packing site is a 1.5m radius sphere, while the robots and boxes are randomly positioned around the packing site. An episode fails if any robot falls or if the environment timeouts. Similar to MRMG navigation, each training episode involves up to four robots and four boxes, with the number of robots and boxes varying across episodes.

We uniformly sample the initial distances of the robots and boxes from the packing site. If the robots successfully push all boxes to the packing site, we increase the upper bounds of the sampling range by 1m. Otherwise, we decrease the upper bounds by 1m. The sampling range is capped between 1.510m for robots and boxes.

For both tasks, we train the policies with 2048 environments using a RTX3090 GPU. The training time is 83 hours for the MRMG navigation task and 15 hours for the box packing task.

E. gRPC Communication Framework

We implement a multi-robot communication framework using the gRPC framework [20], which serializes messages between a server and its clients distributed in different machines. In our case, a server periodically collects the most recent states from all robot clients and sends them to anyone who makes a request. Each robot initiates a client when it starts, requesting other robots' states when needed by its policy. Our server-client communication framework is illustrated in Fig. 4.

IV. EXPERIMENTS AND RESULTS

We validate our approach in a series of hardware and simulation experiments, with the main results showcased in the supplementary video.¹ In all experiments, the robots do not possess the global map or share map information. Instead, each robot only observes a 4.35 m × 2.85 m rectangular local elevation map centered on itself, as shown in all the top-down views in Fig. 5.

¹Supplementary video: <https://youtu.be/uYr9nrFBT-s>

We refer readers to [3] for more details about the elevation map for terrain perception. Note that the blue shaded regions in the top-down views in Fig. 5 are only for visualization and are not related to robot perception.

A. Robot Navigation

We conduct three MRMG navigation experiments using two wheeled-legged quadrupedal robots [21]. In each experiment, the same control policy is used across all robots, and the robots' states are shared through Wi-Fi (see Section III-E).

1) *MRMG Navigation*: Two robots are tasked with reaching three shared goals, as shown in Fig. 5(A-1). The task is completed once all goals are reached by any robot. Each robot strategically chooses the closest goal while leaving farther goals to the other robot (Fig. 5(A-2)). Upon finishing the first two goals (goals 1 and 3 in the figure), both robots try to reach goal 2 and meet at the narrow corridor. At the entrance, the robots avoid collision, and one robot stops and leaves the last goal to the other robot.

The goal prioritization and collision avoidance behavior observed in this experiment is learned, with no handcrafted state machine or task scheduling as seen in [27], [28].

2) *Dead End*: In the experiment shown in Fig. 5(B), two robots are given a single waypoint to reach. Unlike the previous experiment, both robots have to reach the waypoint. This is accomplished by training a policy focused on maximizing the number of robots that successfully reach the goal, with less emphasis on task efficiency. Robot A has an obstacle-free path to the goal in the line of sight, whereas robot B starts from a corridor with a dead end. Notably, neither robot has access to a global map; they rely solely on local perception with a limited field of view.

In the beginning, both robots navigate toward the goal. Then robot B encounters the dead end. Observing robot A's successful path to the goal, the trapped robot B adapts and follows robot A's path, successfully escaping from the dead end. In contrast, Fig. 5(C) shows a single robot scenario. Without its collaborator, robot B fails to find the exit and gets trapped in a local minimum, oscillating back and forth.

This experiment demonstrates the effectiveness of our approach: the control policy observes the other agents' behavior and utilizes this information to resolve navigation challenges.

3) *Multi-Goal Navigation With Obstacles*: Mobile robots are often tasked with multiple waypoints, e.g., path-following or multi-goal delivery tasks. Typically, the goals are given to the robot in a fixed order [21], [27]. Additional state machines or human intervention may be necessary if a waypoint becomes unreachable due to unexpected obstacles or map changes.

Our approach addresses this challenge by letting the policy dynamically determine the visiting order of waypoints. In the experiment in Fig. 5(D), we give a robot multiple intermediate waypoints along a straight path, one of which is blocked by obstacles. The robot navigates to the waypoints sequentially, automatically skipping the blocked waypoint. As shown in

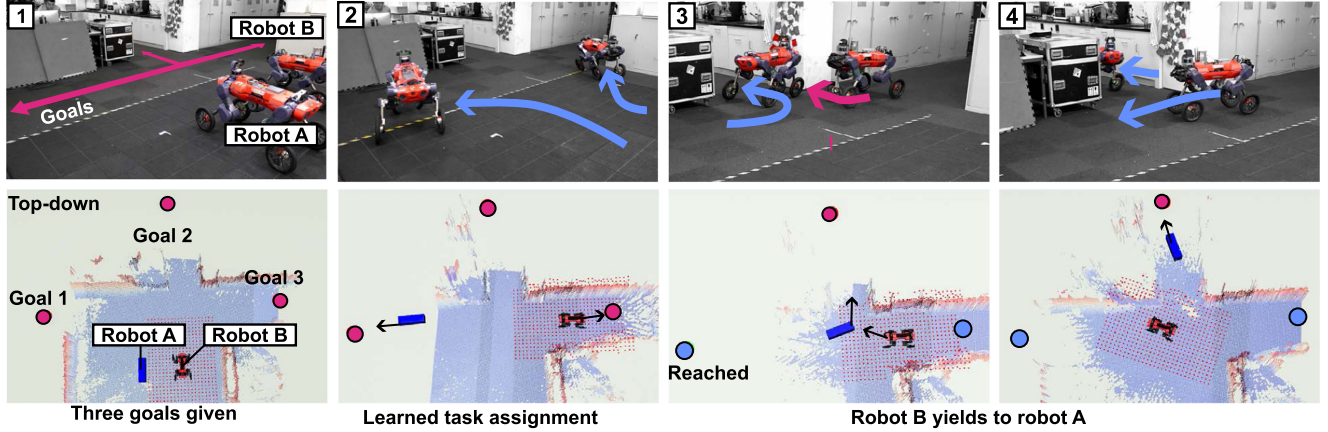
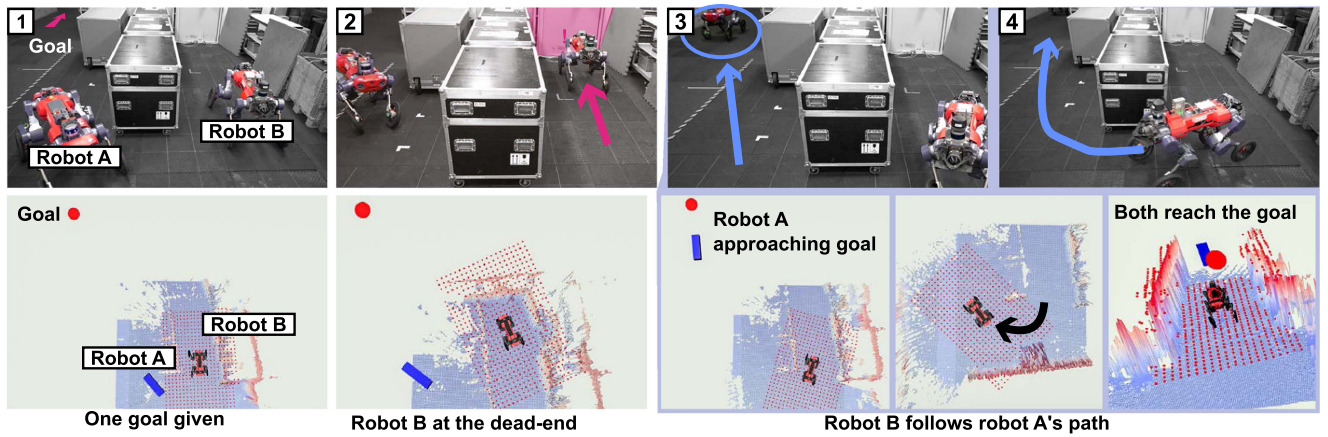
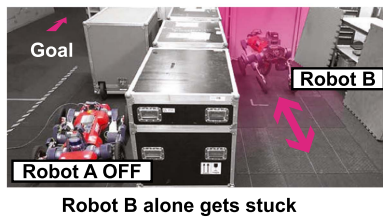
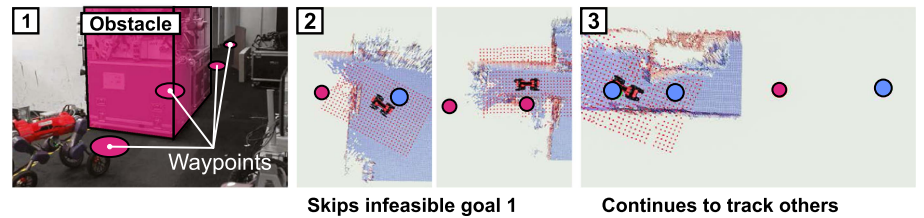
A. Multi-robot multi-goal**B. Dead-end (Two robots)****C. Dead-end (Single robot)****D. Multi-goal navigation**

Fig. 5. Robot experiments with camera views and top-down views. (A) MRMG navigation with two robots and three goals. (B) Dead-end experiment with two robots. (C) Dead-end experiment with a single robot. (D) Single-robot multi-goal navigation.

Fig. 5(D-2) and Fig. 5(D-3), our system can delegate such high-level decision-making challenges to the policy.

B. Learned Collaborative Behaviors

We then analyze the learned collaboration with two experiments on the box packing task. For both experiments, we use the same policy trained following the training procedure in Section III-D2.

1) *Dynamic Prioritization*: We conduct a sensitivity analysis to examine which neighbors and boxes each robot focuses on at each time step. Similarly to [1], we compute the gradient magnitude of the output feature from each permutation invariant encoder with respect to corresponding robot/box states. Intu-

itively, if a neighboring robot or a box does not impact the ego robot's behavior, perturbing it would not change the output of the encoder.

The resulting saliency map in the box packing task is shown in Fig. 6. It is observed that the robots strategically allocate their attention to boxes and adjacent robots that they may interact with. This focus dynamically adapts in real-time to the evolving situation. This dynamic robot/box prioritization is a learned behavior, allowing robots to adjust to changing contexts in cooperation tasks.

2) *Adaptation to Higher Numbers*: We examine whether an increase in the number of robots leads to improved task performance across different numbers of boxes. For this, we execute the same policy as in Section IV-B1 with different combinations of robots and boxes.

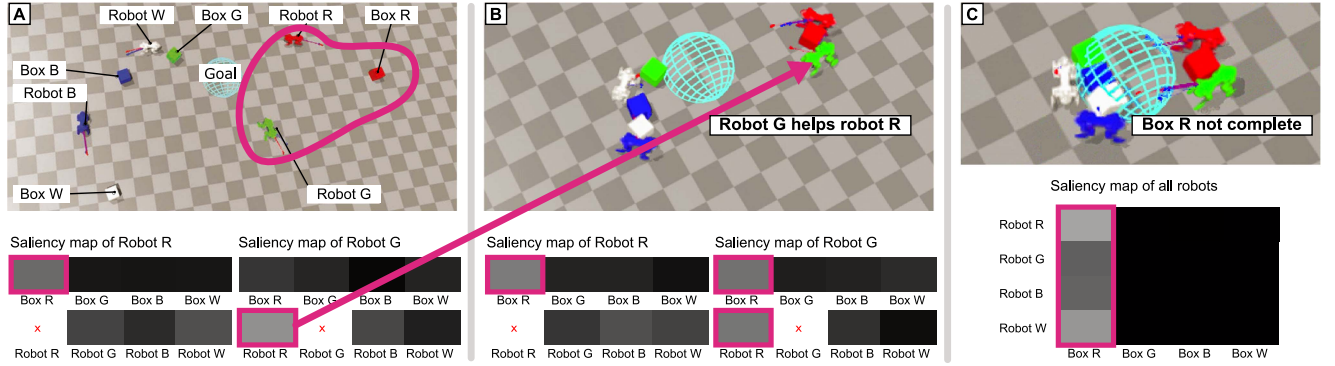


Fig. 6. Saliency maps of two robots during the box packing task. The four robots have to move the boxes into the goal point. (A) At the beginning, robot G does not focus on any box but attends to the states of robot R. (B) Then robot G joins robot R to transport the box R together. (C) Once all boxes are at the goal except for box R, all robots shift their focus to box R.

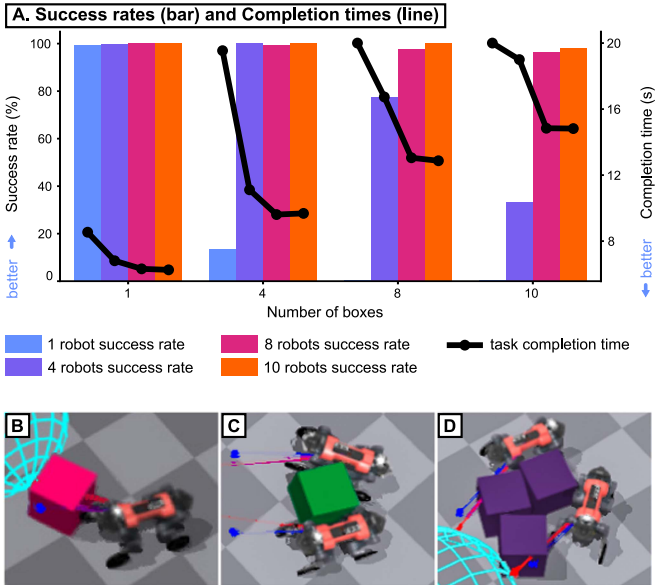


Fig. 7. Impact of robot numbers on box packing tasks. (a) Success rates (bar columns) and Completion times (dotted lines) of 1-10 robots packing 1-10 boxes. (b) 1 robot transports a box by walking sideways. (c) 2 robots are faster by squeezing the box in between them. (d) 2 robots can transport more than 1 box at the same time.

Fig. 7(a) shows the success rate and completion time of different numbers of robots packing boxes within a 20s time limit. We observe that, as the number of robots increases, the task is more likely to succeed and requires less time, especially for higher numbers of boxes.

Notably, the completion time decreases as more robots are involved, even in scenarios with just one box. This tendency persists across all combinations of robot and box numbers once the number of robots surpasses the number of boxes.

The decrease in completion time would not be as significant if each box were always assigned to a single robot. We observe that when one robot pushes one box, it uses its side to avoid the box from slipping away (Fig. 7(b)). The presence of more robots allows for faster transportation by squeezing boxes between them, as shown in Fig. 7(c) and Fig. 7(d).

TABLE II

AVG PERFORMANCE WITH HIGH NUMBERS OF ROBOTS PACKING 10 BOXES

	20 robots	30 robots	40 robots	50 robots
Avg Success Rate	0.94	0.87	0.64	0.52

This example shows the benefit of using RL to learn collaborative behaviors that cannot be easily handcrafted. However, our policy performance eventually decreases when the number of collaborating robots is too high. In the case of packing 10 boxes, we increase the number of robots up to 50 and record the average success rate over 10 experiments. The result is shown in Table II. Note that the policy is trained with only up to 4 robots. it reaches a limit due to the crowded environment and the extrapolation capacity of the policy network.

C. Comparison to Non-Permutation Invariant Encoder

We now assess the role of the permutation invariant encoders in enhancing cooperative task performance. We use a MRMG navigation policy trained based in Section III-D1.

For the baseline policy, we replace our permutation invariant encoder with a modified version of the observation encoder used by [17], where robot/goal states are concatenated with the ego robot's proprioceptive and exteroceptive observations. Unlike their architecture where each robot learns a separate policy network, we train the same policy used across all robots to be consistent with our training pipeline. Since the baseline model does not use a permutation invariant encoder, we manually set the robot/goal order and fix it over the entire episode. Like the training procedure in Section III-D1, we vary the number of robots and goals in different training environments and pad the concatenated features with zeros if the environment has fewer robots/goals than the max numbers (4 robots, 4 goals). Note that the baseline policy cannot handle more than the max numbers but our policy can.

Our policy yields a success rate of 96.7% on the task and an average completion time of 20.3s over all types of terrains, compared to 86.4% and 22.7s for the baseline policy. Fig. 8 shows their different behaviors in one example. In the cross-like corridor environment, our policy exhibits the leader-following

TABLE III
COMPARISON IN CENTROID CONTROL TASK

	Tracking error (m)	Settling time to 0.1 m (s)	Centroid travel distance (m)	Solver time (ms)
Ours 4 robots	0.0168 ± 0.0085	4.70 ± 1.48	10.11 ± 3.03	0.51 ± 0.03
MPC [13] 4 robots	0.0070 ± 0.0030	5.35 ± 0.92	10.38 ± 1.90	9.61 ± 0.29
Ours 10 robots	0.0478 ± 0.0249	8.02 ± 0.78	10.53 ± 1.82	0.77 ± 0.07
MPC [13] 10 robots	0.0044 ± 0.0020	5.90 ± 1.35	11.18 ± 2.44	91.03 ± 5.52

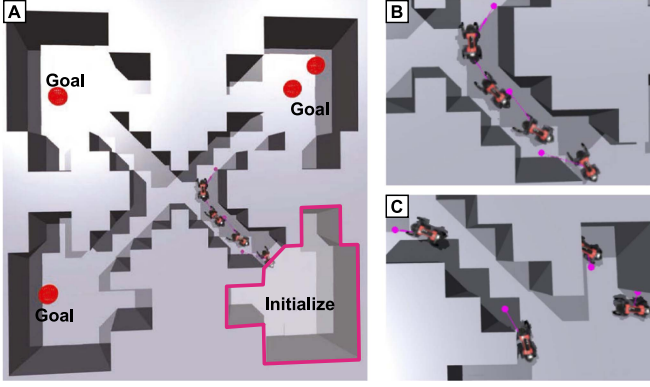


Fig. 8. Comparison with a baseline [17]. (A) MRMG navigation test environment. Four robots are spawned in the bottom right corner, and the goals are sampled from the other three corners. (B) For our policy with permutation invariant encoders, when one robot finds a way out of the local minima, robots exhibit leader-following behavior. (C) The leader-following behavior does not emerge with the baseline policy without permutation invariant encoders.

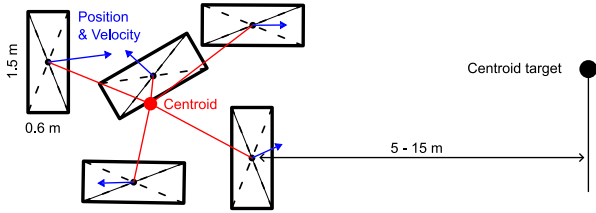


Fig. 9. Two-dimensional centroid control problem. A planner (Ours or MPC) controls the position and velocity of cuboids to let their centroid (red dot) track a target position. The cuboids are randomly spawned between 515 m away from the goal.

behavior in Fig. 8(B). The behavior is also shown in Fig. 5(B). Once one of the robots finds a way out of the first room, the others follow the robot.

Our results show that permutation invariant encoders significantly improve the performance of neighbor-aware policies by effectively aggregating global contextual information.

D. Comparison to an Optimal Control Approach

To assess the optimality of our learned control policy, we compare it against a centralized model predictive control (MPC) approach for the centroid control task depicted in Fig. 9. In this task, the objective is to generate position and velocity targets for each cuboid to move the group's centroid to a goal position while avoiding collisions.

Our policy is trained with 1–6 cuboid agents in each environment and benchmarked against a simplified version of the planner by [13] as the baseline. The control task is formulated as

a kinematic planar system involving R cuboids. Both approaches solve for the desired velocity and position of each cuboid.

1) *MPC Formulation*: We represent each cuboid i with a two-dimensional position $\mathbf{p}_i \in \mathbb{R}^2$ and a yaw angle $\psi_i \in \mathbb{R}$. Given a discrete time horizon $N \in \mathbb{N}$ and a step size $\Delta t > 0$, and having defined the i th cuboid state and input sampled at time step k as $\mathbf{x}_{i,k} := [\mathbf{p}_{i,k}^\top, \psi_{i,k}, \dot{\mathbf{p}}_{i,k}^\top, \dot{\psi}_{i,k}]^\top$ and $\mathbf{u}_{i,k} := [\ddot{\mathbf{p}}_{i,k}^\top, \ddot{\psi}_{i,k}]^\top$, respectively, we formulate our optimal control problem (OCP) as the following nonlinear programming problem:

$$\min_{\mathbf{x}, \mathbf{U}} \sum_{k=0}^{N-1} \left[\|\mathbf{c}_k - \mathbf{c}^*\|_{1, \mathbf{W}_c} + \|\dot{\mathbf{c}}_k - \dot{\mathbf{c}}^*\|_{2, \mathbf{W}_c}^2 + \sum_{i=0}^{R-1} \left[\|\mathbf{x}_{i,k} - \mathbf{x}_{i,k}^*\|_{2, \mathbf{W}_x}^2 + \|\mathbf{u}_{i,k}\|_{2, \mathbf{W}_u}^2 \right] \right]$$

$$\text{s.t. } \mathbf{x}_{i,0} = \mathbf{x}_{i,m}, \forall i,$$

$$\mathbf{x}_{i,k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k, \forall i, \forall k,$$

$$\|\mathbf{p}_{i,k} - \mathbf{p}_{j,k}\| \geq 2\rho, \forall i, \forall j, \forall k,$$

$$|b\dot{\mathbf{p}}_{i,k}^x| \leq v_{\max}^x, |b\dot{\mathbf{p}}_{i,k}^y| \leq v_{\max}^y, |\dot{\psi}_{i,k}| \leq \omega_{\max}, \forall i, \forall k,$$

where $\mathbf{c}^*$ and $\dot{\mathbf{c}}^*$ are the reference centroid position and velocity, respectively, $\mathbf{c}_k := \sum_i \mathbf{p}_{i,k}/R$, while $\mathbf{x}_{i,k}^*$ is the reference state of the i th robot at time step k . $\mathbf{W}_\square$ is a positive definite diagonal matrix for the weighted 1- and 2-norms $\|\cdot\|_{1, \mathbf{W}_\square}$ and $\|\cdot\|_{2, \mathbf{W}_\square}$, respectively; $\mathbf{x}_{i,m}$ is the i th cuboid measured state. We integrate the input accelerations using a semi-implicit Euler method, which translates to the following system matrices:

$$\mathbf{A} := \begin{bmatrix} \mathbf{I}_3 & \Delta t \mathbf{I}_3 \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \mathbf{B} := \begin{bmatrix} \Delta t^2 \mathbf{I}_3 \\ \Delta t \mathbf{I}_3 \end{bmatrix}.$$

2) *Results*: Table III shows the mean and standard deviation of three evaluation metrics for both our decentralized RL policy and the centralized MPC planner on two setups: centroid control with 4 cuboids and 10 cuboids. We run 10 experiments with cuboids randomly positioned 515m away from the goal and compare the final tracking errors, settling times, centroid travel distances, and solver time of both methods on both setups. We remark that the 10-cuboid scenario is not seen during our policy training stage, as the training environments only consist of 1 to 6 cuboids.

Both approaches exhibit similar performance. With 4 cuboids, MPC leads by a narrow margin on the tracking error (~ 1 cm), but it has slightly longer settling times. In all setups, our RL policy showcases shorter centroid travel distances, suggesting more efficient goal-pursuit trajectories. The compared metrics indicate that our decentralized RL policy can find near-optimal solutions in the considered scenario.

Our decentralized policy generalizes to unseen tasks involving 10 cuboids with minimal deterioration from the 4-cuboid setup, with $\sim 3\text{cm}$ higher tracking errors and $\sim 3.3\text{s}$ longer settling times. The increased solver times of the MPC approach (~ 118 times higher than the RL approach with 10 cuboids) highlight the limited scalability of conventional centralized methods. In contrast, our decentralized RL policy maintains nearly constant computation times regardless of the number of robots involved.

V. CONCLUSION

We present a decentralized MARL framework using permutation invariant neural network encoders capable of completing complex MRMG tasks in diverse environments. We validated our approach by implementing and testing a complete system to control a flexible number of wheeled-legged quadrupeds navigating to a flexible number of goals and packing a flexible number of boxes. Through experiments in the real-world and simulation, we show our framework learns a near-optimal policy that enables robots to dynamically prioritize neighbors and goals and can scale to unseen numbers during deployment.

Apart from the limitation of the network capacity, our framework can be limited by scalability issues from communication bandwidth in real-world deployment.

For future work, We plan to extend our research to encompass heterogeneous robots, such as robots equipped with different sensors or manipulators for specialized tasks, and equip robots with on-board semantic vision to sense objects.

ACKNOWLEDGMENT

The authors would like to thank Yifan Liu for the support in implementing simulation environments for the MRMG navigation training. We thank Alexander Reske and Turcan Tuna for helping with the real robot experiments.

REFERENCES

- [1] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Sci. Robot.*, vol. 5, no. 47, 2020, Art. no. eabc5986.
- [2] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conf. Robot Learn.*, 2022, pp. 91–100.
- [3] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Sci. Robot.*, vol. 7, no. 62, 2022, Art. no. eabk2822.
- [4] D. Hoeller, L. Wellhausen, F. Farshidian, and M. Hutter, "Learning a state representation and navigation in cluttered and dynamic environments," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 5081–5088, Jul. 2021.
- [5] C. Chen, J. Frey, P. Arm, and M. Hutter, "SMUG planner: A safe multi-goal planner for mobile robots in challenging environments," *IEEE Robot. Automat. Lett.*, vol. 8, no. 11, pp. 7170–7177, Nov. 2023.
- [6] J. Jiang and Z. Lu, "Learning attentional communication for multi-agent cooperation," *Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [7] Y. Yang, R. Luo, M. Li, M. Zhou, W. Zhang, and J. Wang, "Mean field multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5571–5580.
- [8] Y. Niu, R. R. Paleja, and M. C. Gombolay, "Multi-agent graph-attention communication and teaming," in *Proc. Auton. Agents Multiagent Syst.*, 2021, pp. 964–973.
- [9] I. -J. Liu, R. A. Yeh, and A. G. Schwing, "PIC: Permutation invariant critic for multi-agent deep reinforcement learning," in *Proc. Conf. Robot Learn.*, 2020, pp. 590–602.
- [10] Z. Qin, K. Zhang, Y. Chen, J. Chen, and C. Fan, "Learning safe multi-agent control with decentralized neural barrier certificates," in *Proc. 9th Int. Conf. Learn. Representations*, 2021.
- [11] C. Yang et al., "Collaborative navigation and manipulation of a cable-towed load by multiple quadrupedal robots," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 10041–10048, Oct. 2022.
- [12] R. T. Fawcett, L. Amanzadeh, J. Kim, A. D. Ames, and K. A. Hamed, "Distributed data-driven predictive control for multi-agent collaborative legged locomotion," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 9924–9930.
- [13] F. De Vincenti and S. Coros, "Centralized model predictive control for collaborative loco-manipulation," in *Proc. Robot. Sci. Syst. XIX*, 2023, p. 0 50.
- [14] J. Yang, A. Nakhaei, D. Isele, K. Fujimura, and H. Zha, "CM3: Cooperative multi-goal multi-stage multi-agent reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [15] F. Kennel-Maushart, R. Poranne, and S. Coros, "Interacting with multi-robot systems via mixed reality," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 11633–11639.
- [16] R. Li, A. Jabri, T. Darrell, and P. Agrawal, "Towards practical multi-object manipulation using relational reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 4051–4058.
- [17] Y. Ji, B. Zhang, and K. Sreenath, "Reinforcement learning for collaborative quadrupedal manipulation of a payload over challenging terrain," in *Proc. IEEE 17th Int. Conf. Automat. Sci. Eng.*, 2021, pp. 899–904.
- [18] H. Ha, J. Xu, and S. Song, "Learning a decentralized multi-arm motion planner," in *Proc. Conf. Robot Learn.*, 2021, pp. 103–114.
- [19] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 652–660.
- [20] gRPC, "About grpc," 2023. [Online]. Available: <https://grpc.io/about/>
- [21] M. Bjelonic et al., "Keep rollin'—whole-body motion control and planning for wheeled quadrupedal robots," *IEEE Robot. Automat. Lett.*, vol. 4, no. 2, pp. 2116–2123, Apr. 2019.
- [22] Y. Li et al., "Permutation invariant policy optimization for mean-field multi-agent reinforcement learning: A principled approach," 2021, *arXiv:2105.08268*.
- [23] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3744–3753.
- [24] Y. Tang and D. Ha, "The sensory neuron as a transformer: Permutation-invariant neural networks for reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 22574–22587, 2021.
- [25] S. Liu, G. Lever, J. Merel, S. Tunyasuvunakool, N. Heess, and T. Graepel, "Emergent coordination through competition," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.
- [26] R. Zhang, G. Chen, J. Hou, Z. Li, and A. Knoll, "PIPO: Policy optimization with permutation-invariant constraint for distributed multi-robot navigation," in *Proc. IEEE Int. Conf. Multisensor Fusion Integration Intell. Syst.*, 2022, pp. 1–7.
- [27] F. Schmalstieg, D. Honerkamp, T. Welschehold, and A. Valada, "Learning long-horizon robot exploration strategies for multi-object search in continuous action spaces," in *Proc. Int. Symp. Robot. Res.*, 2022, pp. 52–66.
- [28] Q. Xu, J. Li, S. Koenig, and H. Ma, "Multi-goal multi-agent pickup and delivery," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 9964–9971.
- [29] J. Wu, X. Sun, A. Zeng, S. Song, S. Rusinkiewicz, and T. Funkhouser, "Spatial intention maps for multi-agent mobile manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 8749–8756.
- [30] O. Nachum, M. Ahn, H. Ponte, S. Gu, and V. Kumar, "Multi-agent manipulation via locomotion using hierarchical sim2real," in *Proc. Conf. Robot Learn.*, 2020, pp. 110–121.
- [31] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Sci. Robot.*, vol. 9, no. 89, 2024, Art. no. eadi9641.
- [32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [33] P.-W. Chou, D. Maturana, and S. Scherer, "Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 834–843.